

Contents

The Socket Exchange — Experiment Report	2
1. Implementation Decisions	2
1.1 Concurrency and I/O approach	2
1.2 Why this approach	2
1.3 Other significant design decisions	2
2. Experiment 1 — Listening and Connected Sockets	3
Answer	3
Approach	4
Commands and tools	4
Evidence	4
3. Experiment 2 — Observing TCP Connection States	5
Answer	5
Approach	5
Commands and tools	5
Evidence	6
4. Experiment 3 — TCP as a Byte Stream	7
Answer	7
Approach	7
Commands and tools	7
Evidence	8
5. Experiment 4 — One Client Should Not Stall the Others	9
Answer	9
Approach	9
Commands and tools	9
Evidence	9
6. Experiment 5 — Multiple Clients and I/O Multiplexing (<i>optional</i>)	11
Answer	11
Approach	11
Commands and tools	11
Evidence	11
7. Experiment 6 — FIN vs. RST	13
Answer	13
Approach	13
Commands and tools	13
Evidence	14
8. Experiment 7 — Backpressure and the Slow Receiver	15
8.1 Answer	15
8.2 Pre-flight arithmetic — stated before running	15
8.3 Run A — stock buffers	16
8.4 Runs B and D — shrinking the buffers, twice, with no effect	18
8.5 The correction: buffer limits are not enforced against a non-reading receiver	18
8.6 Run E — forcing backpressure by volume instead	20
8.7 The result that matters: the engine is decoupled	24
8.8 What we would say if asked “did the experiment work?”	24
9. Experiment 8 — Unexpected Client Disconnection	25
Answer	25
Approach	25
Commands and tools	25
Evidence	25
10. Additional observations	27
10.1 Coalescing — the reverse of Experiment 3	27
10.2 errno reference (T12)	27

10.3 Market-Data client commands, completed	28
10.4 Measurement methodology	29
11. Reproducing this work	29

The Socket Exchange — Experiment Report

COL334 Assignment 2 · Amar Sinha (2024CS10388), Ritwik Sehrawat (2024AM10224) Implementation: Python 3, socket + select.kqueue() used directly Testbed: FreeBSD 15.1 VM, all traffic over lo0

1. Implementation Decisions

1.1 Concurrency and I/O approach

A single-threaded event loop over select.kqueue(), called directly. Not selectors (it abstracts over kqueue/epoll/poll and hides which syscall is actually in use), not asyncio (excluded by the assignment), and no third-party networking libraries.

The whole server runs in one thread. kqueue() returns only the subset of registered descriptors that are ready, so the cost of a loop iteration is $O(\text{ready})$, not $O(\text{registered})$.

1.2 Why this approach

Three reasons, in decreasing order of weight.

It answers Experiment 7 structurally rather than with special-case code. The matching engine has no access to a socket. engine.handle() takes a session id and a parsed command and returns a list of (sid, message) pairs; the only function in the program that calls send() is flush(). TCP backpressure therefore has no path by which it could reach the engine. This is a checkable invariant, not an intention:

```
grep -ln 'import socket' src/*.py      # server.py, trader.py, market_data.py only
framing.py, protocol.py and engine.py never import socket.
```

A thread-per-connection design buys nothing in CPython. It would need two threads per connection (a blocking reader and a writer draining a queue), and because the GIL serialises bytecode there is no CPU parallelism to gain in exchange for the context switches and lock traffic. The matching engine here is two instruments with exact-price dictionary lookups — there is nothing to parallelise even in principle.

No locks means no races. With one thread there is no shared mutable state between concurrent flows of control, so a whole class of defect cannot occur.

The measured contrast is in Experiment 4: the same harness run against a deliberately-built blocking control server times out after 5.105 s with the server parked in sbwait, while this design answers in 0.001 s parked in kqread.

1.3 Other significant design decisions

Per-connection userspace output buffer with lazily-registered write notification. Each connection owns a bytearray (wbuf) plus an offset. queue_out() appends and opportunistically calls flush(); flush() writes with send() until the socket returns BlockingIOError, at which point it records the event, compacts the buffer, and registers

KQ_FILTER_WRITE for that connection only. The write filter is *disabled* again the moment the buffer drains — KQ_FILTER_WRITE is level-triggered, so leaving it armed on an empty buffer is a 100 %-CPU spin.

Deferred close. A socket is never closed inside an event batch. `kevent()` returns up to 256 events at once; closing fd 12 mid-batch would let the kernel immediately reassign fd 12 to a new `accept()`, and a stale event for the old fd 12 would then land on the new connection. Connections are marked dead and appended to a reap list; the actual `close()` happens after the batch.

Orders reference sessions by integer id, not by connection object. `Order.owner_sid` is an int. This is the entire mechanism by which resting orders outlive a TCP disconnect: teardown removes the session from `by_sid` but deliberately does not touch orders or `level`, so a later match still executes and only the departed owner's private notification is dropped. It also prevents a subtler bug — usernames are unique only among *connected* traders, so if orders referenced the username, a new client logging in as *alice* would inherit the previous *alice*'s notifications. Session ids are monotonic and never reused.

Bounded output with a stated policy. A permanently slow consumer is disconnected once its backlog exceeds `EXCH_WBUF_MAX` (default 4 MiB) with reason `slow_consumer` — the “slow consumer disconnect” behaviour real market-data feeds use. The alternative of unbounded buffering is a denial-of-service vector; conflation is wrong here because each TRADE is a distinct event, not a snapshot.

No idle timeouts, anywhere. Experiments 1, 2, 5 and 6 use connections that send nothing at all, and Experiment 4 parks a partial line indefinitely. A login or handshake timeout would fail experiments a correct server passes.

Strict inbound validation, in bytes end to end. Quantities and prices are matched against an anchored `re.fullmatch(rb'[0-9]+')` rather than passed to `int()`, which would otherwise accept `+5, 1_000` (PEP 515 underscores) and `" 5"`. Staying in bytes also removes the Unicode digit problem, where `"0".isdigit()` is `True`.

TCP_NODELAY on accepted sockets, so notification latency is not held hostage to Nagle. This has a visible consequence in Experiment 7 (below): every 17-byte TRADE becomes its own segment.

Graceful QUIT, and where `shutdown()` genuinely belongs. §2.2.5 lists QUIT for both client roles, and §4.1 requires direct use of `shutdown()`. On the client, QUIT calls `s.shutdown(socket.SHUT_WR)` rather than `close()` — a real half-close, so the socket keeps reading until the server closes from its end and nothing already in flight is discarded. On the server, QUIT is intercepted in `server.py` *before* `engine.handle()` is called and is never routed through the engine — the engine must not learn that connections exist at all, which is the same invariant Experiment 7 rests on. If the connection has a userspace backlog, the socket is marked quitting and the actual close is deferred until `flush()` finishes draining it, so a QUIT immediately after a large order burst cannot truncate replies the client was already owed. This is exercised offline by `src/tools/verify_quit.py` (17/17 checks, including the deferred-close case under a forced backlog) and cross-checked against the existing Experiment-7 write-path verifier (19/19, no regression).

2. Experiment 1 — Listening and Connected Sockets

Answer

The listening socket is a *factory*: it has a wildcard foreign address `(*:*)`, sits in state `LISTEN`, and never carries application data. Each accepted connection is a **separate file descriptor**

with a fully specified 4-tuple in state ESTABLISHED. Both share the same local port — it is the 4-tuple, not the port, that identifies a connection.

Approach

Run the harness, then inspect the server's sockets from a second terminal while the experiment client sits idle, correlating the OS view against the server's own trace.

Commands and tools

```
python3 experiment.py 1
sockstat -4 | grep 5000
netstat -an -p tcp | grep 5000
```

Evidence

```
root@freebsd:~/col334_a2 # sockstat -4 | grep 5000
root python3.12 11421 3 tcp4 127.0.0.1:5000 *: *
root python3.12 11421 5 tcp4 127.0.0.1:5000 127.0.0.1:27445
root python3.12 11420 3 tcp4 127.0.0.1:27445 127.0.0.1:5000
root@freebsd:~/col334_a2 # netstat -an -p tcp | grep 5000
tcp4 0 0 127.0.0.1:5000 127.0.0.1:27445 ESTABLISHED
tcp4 0 0 127.0.0.1:27445 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 *.* LISTEN
```

Screenshot 1a — sockstat and netstat, both taken while the harness is paused mid-experiment: one LISTEN row plus one ESTABLISHED pair.

```
$ sockstat -4 | grep 5000
root python3.12 4898 3 tcp4 127.0.0.1:5000 *: *
root python3.12 4898 5 tcp4 127.0.0.1:5000 127.0.0.1:28907
root python3.12 4897 3 tcp4 127.0.0.1:28907 127.0.0.1:5000
```

```
$ netstat -an -p tcp | grep 5000
tcp4 0 0 127.0.0.1:5000 127.0.0.1:28907 ESTABLISHED
tcp4 0 0 127.0.0.1:28907 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 *.* LISTEN
```

Server trace for the same run:

```
0.184ms listen fd=3 host='127.0.0.1' port=5000 backlog=4096
76.643ms accept sid=1 fd=5 peer='127.0.0.1:36613' nconn=1
76.682ms eof_rst sid=1 fd=5 errno=54 ev_fflags=54
76.861ms accept sid=2 fd=5 peer='127.0.0.1:28907' nconn=1
```

Table T1

fd	role	local	foreign	state
3	listening	127.0.0.1:5000	*.*	LISTEN
5 (sid 2)	connected	127.0.0.1:5000	127.0.0.1:28907	ESTABLISHED

Note that fd 3 is never assigned a session id — it is not a conversation. Note also that the harness's readiness probe (sid 1) arrives as an abortive RST *before* the experiment proper: errno=54 with the same value visible in ev_fflags, which matters in Experiment 6.

3. Experiment 2 — Observing TCP Connection States

Answer

LISTEN → three-way handshake → ESTABLISHED at both ends. The client closes first, sending FIN; the server enters **CLOSE_WAIT** and the client FIN_WAIT_2. The server's `recv()` returns `b''`, it closes, and the connection moves through LAST_ACK to CLOSED. The client, as the active closer, holds TIME_WAIT.

The defect graders look for is a server that ignores EOF and leaves sockets in CLOSE_WAIT forever, leaking descriptors. **This server's CLOSE_WAIT lasts 504 μ s.**

Approach

Capture the whole connection lifecycle on the wire, and poll netstat at each phase. Because the state transitions turn out to be far faster than a polling interval, the packet capture — not netstat — is what actually establishes the sequence.

Commands and tools

```
tcpdump -i lo0 -n -s0 -w exp2.pcap 'tcp port 5000' &  
python3 experiment.py 2  
netstat -an -p tcp | grep 5000          # repeatedly, at each phase  
tcpdump -r exp2.pcap -n -ttt -S
```

Evidence

[illegible]

Screenshot 2a — netstat sampled repeatedly across both phases: the pair of ESTABLISHED rows while the client is connected, collapsing to LISTEN-only once it closes.

```

root@freebbsd:/col334_a2 # kill %1
root@freebbsd:/col334_a2 # 13 packets captured
13 packets received by filter
0 packets dropped by kernel

tcpdump -r exp2.pcap -n -ttt -S
reading from file exp2.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000000 IP 127.0.0.1.32960 > 127.0.0.1.5000: Flags [S], seq 642414569, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 845762809 ecr 0], length 0
00:00:00.000026 IP 127.0.0.1.5000 > 127.0.0.1.32960: Flags [R], seq 0, ack 642414570, win 0, length 0
00:00:00.100012 IP 127.0.0.1.21441 > 127.0.0.1.5000: Flags [S], seq 1671568193, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2949774629 ecr 0], length 0
00:00:00.000046 IP 127.0.0.1.5000 > 127.0.0.1.21441: Flags [S], seq 1565152674, ack 1671568194, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1188180807 ecr 2949774629], length 0
00:00:00.000018 IP 127.0.0.1.21441 > 127.0.0.1.5000: Flags [S], seq 1565152675, win 320, options [nop,nop,TS val 2949774629 ecr 1188180807], length 0
00:00:00.000046 IP 127.0.0.1.5000 > 127.0.0.1.21441: Flags [R], seq 0, ack 1565152675, win 0, options [nop,nop,TS val 2949774629 ecr 1188180807], length 0
00:00:00.000098 IP 127.0.0.1.62724 > 127.0.0.1.5000: Flags [S], seq 2858073199, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3862054566 ecr 0], length 0
00:00:00.000018 IP 127.0.0.1.5000 > 127.0.0.1.62724: Flags [S], seq 84127483, ack 2858073200, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1029497203 ecr 3862054566], length 0
00:00:00.000007 IP 127.0.0.1.62724 > 127.0.0.1.5000: Flags [R], seq 84127483, win 320, options [nop,nop,TS val 3862054566 ecr 1029497203], length 0
00:00:10.100375 IP 127.0.0.1.62724 > 127.0.0.1.5000: Flags [F], seq 2058073200, ack 84127483, win 320, options [nop,nop,TS val 3862064574 ecr 1029497203], length 0
00:00:00.000030 IP 127.0.0.1.5000 > 127.0.0.1.62724: Flags [S], seq 2058073201, win 320, options [nop,nop,TS val 1029507211 ecr 3862064574], length 0
00:00:00.000004 IP 127.0.0.1.5000 > 127.0.0.1.62724: Flags [F], seq 84127483, ack 2058073201, win 320, options [nop,nop,TS val 1029507211 ecr 3862064574], length 0
00:00:00.000014 IP 127.0.0.1.62724 > 127.0.0.1.5000: Flags [F], seq 84127484, win 320, options [nop,nop,TS val 3862064574 ecr 1029507211], length 0
[1] Done
root@freebbsd:/col334_a2 # tcpdump -i lo0 -n -s0 -w exp2.pcap tcp port 5000

```

Screenshot 2b — exp2.pcap readback: the complete four-way close.

```
+0.100912s    SYN 21441->5000  -> SYN/ACK -> ACK -> RST from 21441    <-
readiness probe
+0.000098s    SYN 62724->5000  -> SYN/ACK -> ACK                      <- real client
+10.003715s   [62724->5000] FIN,ACK          <- client closes (active closer)
```

+0.000030s	[5000->62724] ACK	<- server ACKs; CLOSE_WAIT begins
+0.000504s	[5000->62724] FIN,ACK	<- server's own FIN; CLOSE_WAIT = 504 us
+0.000014s	[62724->5000] ACK	<- client -> TIME_WAIT

Table T2

time	event	client state	server state
t ₀	handshake completes	ESTABLISHED	ESTABLISHED
t ₀ +10 s	client close() → FIN	FIN_WAIT_1→FIN_WAIT_2	CLOSE_WAIT
+30 μs	server ACKs the FIN	FIN_WAIT_2	CLOSE_WAIT
+534 μs	server closes → FIN	TIME_WAIT	LAST_ACK
+548 μs	client ACKs	TIME_WAIT	CLOSED

Measurement note. CLOSE_WAIT was measured five times across independent runs: 656 μs, 507 μs, 609 μs, 984 μs, 504 μs (the last being the captured run shown above). Consistently sub-millisecond, so the transient-CLOSE_WAIT claim is a property of the implementation rather than one lucky sample.

An honest limitation. netstat polling at ~0.5 s never caught CLOSE_WAIT or TIME_WAIT at all — the samples go straight from ESTABLISHED to LISTEN-only. We verified `net.inet.tcp.nolocaltimewait` is 0, so the loopback fast-clear optimisation is *not* what is happening, and with `mss=30000` a real TIME_WAIT should persist ~60 s. We therefore do not claim a mechanism: the states are established by packet capture, and the sampling interval was simply too coarse relative to the transitions to distinguish “a very brief real TIME_WAIT” from “none”.

4. Experiment 3 — TCP as a Byte Stream

Answer

The message arrives in **four pieces**. The harness sends LOGIN, experiment, _trader, \n — 6, 10, 7 and 1 bytes, 0.2 s apart — and the server performs four separate `recv()` calls, emitting a parsed message only on the fourth. Segment boundaries are an artefact of *when the sender wrote*; they carry no application meaning. Only the \n delimiter does.

Approach

Instrument every `recv()` with its byte count and the buffer state before and after, then confirm against the wire that four distinct segments really were transmitted.

Commands and tools

```
tcpdump -i lo0 -n -s0 -w exp3.pcap 'tcp port 5000' &
python3 experiment.py 3
tcpdump -r exp3.pcap -n -ttt -S
```

Evidence

```
root@freebsd:~/col334_a2 # tcpdump -i lo0 -n -s0 -w exp3.pcap 'tcp port 5000' &
root@freebsd:~/col334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
python3 experiment.py 3
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 11319).
  0.113ms listen          fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  80.007ms accept         sid=1 fd=5 peer='127.0.0.1:19220' nconn=1 sndbuf=49032 rcvbuf=81720

Experiment client: connected from 127.0.0.1:65502
  80.088ms accept         sid=2 fd=6 peer='127.0.0.1:65502' nconn=2 sndbuf=49032 rcvbuf=81720

Sending one application-level message using several separate TCP writes...

Sent 6 bytes.
  80.140ms eof_rst        sid=1 fd=5 errno=54 ev_fflags=54
  80.173ms close          sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  80.191ms destroy        sid=1 fd=5 nconn=1
  80.220ms rcv            sid=2 fd=6 n=6 rbuf_before=0 rbuf_after=6 lines_out=0 hex='4c4f47494e20'
Sent 10 bytes.
  281.372ms rcv          sid=2 fd=6 n=10 rbuf_before=6 rbuf_after=16 lines_out=0 hex='6578706572696d6574'
Sent 7 bytes.
  483.431ms rcv          sid=2 fd=6 n=7 rbuf_before=16 rbuf_after=23 lines_out=0 hex='5f747261646572'
Sent 1 bytes.
  684.034ms rcv          sid=2 fd=6 n=1 rbuf_before=23 rbuf_after=0 lines_out=1 hex='0a'
  684.199ms engine       sid=2 n_msgs=1 engine_ns=25458
  684.244ms queue_out    sid=2 fd=6 n=3 pending=3 hwm=3

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
^C
Experiment finished.
Stopping Exchange Server...
  4924.844ms eof_fin      sid=2 fd=6 ev_eof=True
  4925.175ms signal       signo=15
root@freebsd:~/col334_a2 #
```

Screenshot 3a — the server's own trace: four rcv calls, a parsed message emitted only on the fourth.

```
root@freebsd:~/col334_a2 # kill %1
root@freebsd:~/col334_a2 # 22 packets captured
22 packets received by filter
0 packets dropped by kernel
tcpdump -r exp3.pcap -n -ttt -S
reading from file exp3.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000000 IP 127.0.0.1.26753 > 127.0.0.1.5000: Flags [S], seq 410569880, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1182648609 ecr 0], length 0
00:00:00.000029 IP 127.0.0.1.5000 > 127.0.0.1.26753: Flags [R], seq 0, ack 410569881, win 0, length 0
00:00:00.102483 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [S], seq 152221514, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3191004455 ecr 0], length 0
00:00:00.000043 IP 127.0.0.1.5000 > 127.0.0.1.19220: Flags [S.], seq 26693308528, ack 152221515, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2622429932 ecr 3191004455], length 0
00:00:00.000013 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [L], ack 26693308529, win 320, options [nop,nop,TS val 3191004455 ecr 2622429932], length 0
00:00:00.000062 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [R.], seq 152221515, ack 26693308529, win 0, options [nop,nop,TS val 3191004455 ecr 2622429932], length 0
00:00:00.000066 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [S], seq 2692795946, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3532811570 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [S.], seq 1154791355, ack 2692795947, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1455016221 ecr 3532811570], length 0
00:00:00.000006 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [L.], ack 1154791356, win 320, options [nop,nop,TS val 3532811570 ecr 1455016221], length 0
00:00:00.000062 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795947:2692795953, ack 1154791356, win 320, options [nop,nop,TS val 3532811570 ecr 1455016221], length 6
00:00:00.038466 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [L.], ack 2692795953, win 320, options [nop,nop,TS val 1455016200 ecr 3532811570], length 0
00:00:00.162590 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795953:2692795965, ack 1154791356, win 320, options [nop,nop,TS val 3532811766 ecr 1455016200], length 10
00:00:00.037511 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [L.], ack 2692795965, win 320, options [nop,nop,TS val 1455016460 ecr 3532811766], length 0
00:00:00.164443 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795963:2692795970, ack 1154791356, win 320, options [nop,nop,TS val 3532811974 ecr 1455016460], length 7
00:00:00.041018 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [L.], ack 2692795970, win 320, options [nop,nop,TS val 1455016666 ecr 3532811974], length 0
00:00:00.159561 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795970:2692795971, ack 1154791356, win 320, options [nop,nop,TS val 3532812174 ecr 1455016666], length 1
00:00:00.000592 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [P.], seq 1154791356:1154791359, ack 2692795971, win 320, options [nop,nop,TS val 1455016825 ecr 3532812174], length 3
00:00:00.039221 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [L.], ack 1154791359, win 320, options [nop,nop,TS val 3532812214 ecr 1455016825], length 0
00:00:00.208043 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [F.], seq 2692795971, ack 1154791359, win 320, options [nop,nop,TS val 3532816414 ecr 1455016825], length 0
00:00:00.000322 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [L.], ack 2692795972, win 320, options [nop,nop,TS val 1455021065 ecr 3532816414], length 0
00:00:00.000626 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [F.], seq 1154791359, ack 2692795972, win 320, options [nop,nop,TS val 1455021065 ecr 3532816414], length 0
00:00:00.000136 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [L.], ack 1154791360, win 320, options [nop,nop,TS val 3532816414 ecr 1455021065], length 0
[!] Done
tcpdump -i lo0 -n -s0 -w exp3.pcap tcp port 5000
root@freebsd:~/col334_a2 #
```

Screenshot 3b — exp3.pcap readback: four PSH segments of 6/10/7/1 bytes.

```
80.220ms rcv sid=2 fd=6 n=6 rbuf_before=0 rbuf_after=6 lines_out=0 hex='4c4f47494e20'
281.372ms rcv sid=2 fd=6 n=10 rbuf_before=6 rbuf_after=16 lines_out=0 hex='6578706572696d65'
483.431ms rcv sid=2 fd=6 n=7 rbuf_before=16 rbuf_after=23 lines_out=0 hex='5f747261646572'
684.034ms rcv sid=2 fd=6 n=1 rbuf_before=23 rbuf_after=0 lines_out=1 hex='0a'
684.199ms engine sid=2 n_msgs=1 engine_ns=25458
684.244ms queue_out sid=2 fd=6 n=3 pending=3 hwm=3
```

```
+0.000062s PSH 65502->5000 len=6 "LOGIN "
+0.162590s PSH 65502->5000 len=10 "experiment"
+0.164443s PSH 65502->5000 len=7 "trader"
+0.159561s PSH 65502->5000 len=1 "\n"
+0.000592s PSH 5000->65502 len=3 "OK\n"
```

Table T3 — rcv() ledger

#	t (ms)	bytes	payload	rbuf after	messages emitted
1	80	6	LOGIN	6	0
2	281	10	experiment	16	0
3	483	7	_trader	23	0
4	684	1	\n	0	1

Generality (T4). One run proves one case. The framer is additionally tested offline against every split position of a multi-message byte stream (47 / 47 positions identical) and 1,000 random multi-way fragmentations (1000 / 1000 identical) — `tests/test_all.py::test_frag_all_split_positions` and `::test_frag_random_multiway_splits`. Reassembly is therefore not dependent on this run’s particular boundaries.

The reverse direction is covered in §10.1: four application messages arriving in a *single* segment.

5. Experiment 4 — One Client Should Not Stall the Others

Answer

No. A silent client cannot stall the server, because the server never blocks on any individual socket. It is parked in `kevent()`, which reports only descriptors that are ready; a client that sends nothing simply never becomes ready. Client 2 is served in **0.003 s** while Client 1 sits mid-message forever.

Approach

The decisive evidence is one word of kernel state. We built a deliberately naive blocking server as an A/B control and ran the *same* harness experiment against both, comparing `wchan` — the kernel wait channel the process is sleeping on.

Commands and tools

```
python3 experiment.py 4                # real server
EXCH_NAIVE=1 python3 experiment.py 4    # naive blocking control
pgrep -f server.py
ps -o pid,tid,wchan,state -H -p <pid>
netstat -an -p tcp | grep 5000
```

Evidence

```
root@freebsd:~/col334_a2 # pgrep -f server.py
11492
root@freebsd:~/col334_a2 #
root@freebsd:~/col334_a2 # ps -o pid,tid,wchan,state -H -p 11492
  PID   LWP  WCHAN  STAT
11492 100187 kqread Is
root@freebsd:~/col334_a2 #
```

Screenshot 4a — real server, captured from a second terminal while Client 2’s reply is still pending: `ps -o pid,tid,wchan,state` reads `WCHAN=kqread`.

```

root@freebsd:~/col334_a2 # pgrep -f server.py
11499
root@freebsd:~/col334_a2 # pgrep -f server.py 11499
11499
root@freebsd:~/col334_a2 # ps -o pid,tid,wchan,state -H -p 11499
  PID    LWP  WCHAN  STAT
11499 100187 sbwait  Is
root@freebsd:~/col334_a2 # █

```

Screenshot 4b — naive blocking control, same command, same moment in the run: *WCHAN=sbwait*.

Real server (PID 11326):

```

80.128ms   recv sid=2 fd=5 n=20 rbuf_before=0 rbuf_after=20 lines_out=0    <-
"LOGIN blocked_client", no \n
2081.392ms accept sid=3 fd=6 peer='127.0.0.1:16102' nconn=2
2082.615ms recv sid=3 fd=6 n=20 rbuf_before=0 rbuf_after=0 lines_out=1
2082.864ms engine sid=3 n_msgs=1 engine_ns=38459
2083.219ms queue_out sid=3 fd=6 n=3 pending=3 hwm=3
Client 2 response: 'OK'
Elapsed time: 0.003 seconds

```

Naive control (PID 11331):

```

Client 1: connected from 127.0.0.1:55848
Client 1 will send an incomplete application message.
Client 1 will now remain silent.
Client 2: connected from 127.0.0.1:21907
Client 2 response: None
Elapsed time: 5.072 seconds

```

The wchan comparison that decides the experiment is captured separately in Screenshot 4c, since it must be read from a second terminal while each server process is still alive.

Table T5

	real server (kqueue)	naive control (blocking)
wchan	kqread	sbwait
Client 2 response	OK	None
Elapsed	0.003 s	5.072 s (harness timeout)
Client 1 Recv-Q	0	0

sbwait means “blocked waiting on a socket buffer” — the naive server is inside a `recv()` on Client 1’s dead connection and structurally cannot reach `accept()`. *kqread* means “blocked in `kevent()`”, waiting on *any* of many descriptors. That single word is the architectural proof.

A subtlety worth stating. Client 1’s Recv-Q is **0**, not 20. The 20 bytes are not sitting in the kernel — the server already read them, and they are in *its* reassembly buffer awaiting a delimiter. Recv-Q=0 together with `rbuf_after=20` in the trace is what tells the whole story.

6. Experiment 5 — Multiple Clients and I/O Multiplexing (*optional*)

Answer

Connected $\neq$ ready. All five sockets are ESTABLISHED for the entire run, but the server calls `recv()` on exactly three of them — the three that sent data. The two idle connections never generate a readiness event, so the cost of a loop iteration is $O(\text{ready}) = 3$, not $O(\text{registered}) = 5$.

Approach

Identify which descriptors the event loop actually touched (from the trace), then confirm at OS level that the untouched ones were genuinely idle rather than merely unserved.

Commands and tools

```
python3 experiment.py 5
netstat -an -p tcp | grep '\.5000 '      # sampled 4x across the observation window
```

Evidence

```
root@freebsd:~/col1334_a2 # python3 experiment.py 5
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===

Started Exchange Server (PID 11341).
  0.118ms listen      fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  79.138ms accept      sid=1 fd=5 peer='127.0.0.1:28635' nconn=1 sndbuf=49032 rcvbuf=81720
  79.173ms eof_rst      sid=1 fd=5 errno=54 ev_fflags=54
  79.188ms close      sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  79.204ms destroy      sid=1 fd=5 nconn=0

Creating five simultaneous TCP connections...

Client 1: connected from 127.0.0.1:11117
  79.396ms accept      sid=2 fd=5 peer='127.0.0.1:11117' nconn=1 sndbuf=49032 rcvbuf=81720
Client 2: connected from 127.0.0.1:48451
  79.554ms accept      sid=3 fd=6 peer='127.0.0.1:48451' nconn=2 sndbuf=49032 rcvbuf=81720
Client 3: connected from 127.0.0.1:61204
  79.651ms accept      sid=4 fd=7 peer='127.0.0.1:61204' nconn=3 sndbuf=49032 rcvbuf=81720
Client 4: connected from 127.0.0.1:20623
  79.688ms accept      sid=5 fd=8 peer='127.0.0.1:20623' nconn=4 sndbuf=49032 rcvbuf=81720
Client 5: connected from 127.0.0.1:46858
  79.706ms accept      sid=6 fd=9 peer='127.0.0.1:46858' nconn=5 sndbuf=49032 rcvbuf=81720
Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
  79.798ms recv      sid=2 fd=5 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f310a'
  79.890ms engine      sid=2 n_msgs=1 engine_ns=5375
  79.899ms queue_out      sid=2 fd=5 n=3 pending=3 hwm=3
Client 3: sent application data.
  581.748ms recv      sid=4 fd=7 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f330a'
  582.189ms engine      sid=4 n_msgs=1 engine_ns=27458
  582.255ms queue_out      sid=4 fd=7 n=3 pending=3 hwm=3
Client 5: sent application data.
  1083.395ms recv      sid=6 fd=9 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f350a'
  1083.641ms engine      sid=6 n_msgs=1 engine_ns=27333
  1083.682ms queue_out      sid=6 fd=9 n=3 pending=3 hwm=3

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
  16586.312ms signal      signo=15
root@freebsd:~/col1334_a2 #
```

Screenshot 5a — five accepted connections, but recv fires on only three of them; sid 3 and sid 5 never appear.

[illegible]

Screenshot 5b — netstat across five samples: all five connections ESTABLISHED throughout, Recv-Q non-zero only for the three that were sent a reply.

Across the ~16.5 s run, recv fires exactly three times:

```
79.798ms  recv sid=2 fd=5 n=15 lines_out=1 hex='4c4f47494e20636c69656e745f310a' "LOGIN clie
581.748ms  recv sid=4 fd=7 n=15 lines_out=1 hex='4c4f47494e20636c69656e745f330a' "LOGIN clie
1083.395ms recv sid=6 fd=9 n=15 lines_out=1 hex='4c4f47494e20636c69656e745f350a' "LOGIN clie
```

fds 6 and 8 (sid 3 and 5) never appear.

Table T6

sid	fd	peer port	client	appears in any recv?	Recv-Q (both directions)
2	5	11117	1	✓	3 (unread 0K\n at client)
3	6	48451	2	✗	0 / 0
4	7	61204	3	✓	3
5	8	20623	4	✗	0 / 0
6	9	46858	5	✓	3

The mapping was cross-checked two independent ways: by peer port against the harness's own output, and by decoding each recv's hex payload. Recv-Q=0 in *both* directions for clients 2 and 4 is the OS-level confirmation that idle means idle — no bytes were exchanged, as opposed to bytes sent but unread.

7. Experiment 6 — FIN vs. RST

Answer

FIN is in-band and sequenced: it means “no more data from me”, occupies a sequence number, allows data already sent to be delivered, permits half-close, and costs the active closer a TIME_WAIT. It surfaces to the application as `recv()` returning `b''` — end of file, not an error.

RST is out-of-band and unsequenced: it aborts immediately, discards queued data in both directions, produces no TIME_WAIT, and surfaces as `ConnectionResetError` (errno 54).

Approach

Run both terminations, then distinguish them at three levels: the wire, the kernel's socket table, and — the sharpest of the three — the readiness notification itself, before `recv()` is ever called.

Commands and tools

```
tcpdump -i lo0 -n -s0 -w exp6.pcap 'tcp port 5000' &
python3 experiment.py 6
netstat -an -p tcp | grep '\.5000 '
tcpdump -r exp6.pcap -n -ttt -S
```

Evidence

```
root@freebsd:~/col334_a2 # netstat -an -p tcp | grep '\.5000'
root@freebsd:~/col334_a2 # kill %1
root@freebsd:~/col334_a2 # 34 packets captured
34 packets received by filter
0 packets dropped by kernel
tcpdump -r exp6.pcap -n -ttt -S
reading from file exp6.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000000 IP 127.0.0.1.57843 > 127.0.0.1.5000: Flags [S], seq 672047514, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3215072442 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.57843: Flags [R.], seq 0, ack 672047515, win 0, length 0
00:00:00.104502 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [S], seq 2836097739, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 550528494 ecr 0], length 0
00:00:00.000037 IP 127.0.0.1.5000 > 127.0.0.1.12541: Flags [S.], seq 903231691, ack 2836097740, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3047162408 ecr 550528494], length 0
00:00:00.000013 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [F.], ack 903231692, win 320, options [nop,nop,TS val 550528494 ecr 3047162408], length 0
00:00:00.000073 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [R.], seq 2836097740, ack 903231692, win 0, options [nop,nop,TS val 550528494 ecr 3047162408], length 0
00:00:00.000195 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [S], seq 4042106588, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2731617098 ecr 0], length 0
00:00:00.000004 IP 127.0.0.1.5000 > 127.0.0.1.22932: Flags [S.], seq 1104886721, ack 4042106587, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 742386677 ecr 2731617098], length 0
00:00:00.000005 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], ack 1104886720, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:00.000030 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], seq 4042106587, ack 1104886720, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:00.000082 IP 127.0.0.1.5000 > 127.0.0.1.22932: Flags [F.], ack 4042106588, win 320, options [nop,nop,TS val 742386677 ecr 2731617098], length 0
00:00:00.000104 IP 127.0.0.1.5000 > 127.0.0.1.22932: Flags [F.], seq 1104886720, ack 4042106588, win 320, options [nop,nop,TS val 742386677 ecr 2731617098], length 0
00:00:00.000082 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], ack 1104886721, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:10.003722 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [S], seq 3508927132, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3479980373 ecr 0], length 0
00:00:00.000217 IP 127.0.0.1.5000 > 127.0.0.1.34505: Flags [S.], seq 2987702093, ack 3508927133, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 794655570 ecr 0], length 0
00:00:00.000039 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [F.], ack 2987702094, win 320, options [nop,nop,TS val 3479980373 ecr 794655570], length 0
00:00:00.000018 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [R.], seq 3508927133, ack 2987702094, win 0, options [nop,nop,TS val 3479980373 ecr 794655570], length 0
00:00:14.312347 IP 127.0.0.1.39505 > 127.0.0.1.5000: Flags [S], seq 2808368063, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2808249619 ecr 0], length 0
00:00:00.000026 IP 127.0.0.1.5000 > 127.0.0.1.39505: Flags [R.], seq 0, ack 2808368064, win 0, length 0
00:00:00.101961 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [S], seq 3200946643, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 991525530 ecr 0], length 0
00:00:00.000034 IP 127.0.0.1.16177 > 127.0.0.1.45901: Flags [S.], seq 5248096697, ack 3200946644, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2000548351 ecr 991525530], length 0
00:00:00.000012 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [F.], ack 5248096698, win 320, options [nop,nop,TS val 991525530 ecr 2000548351], length 0
00:00:00.000063 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [F.], seq 3200946644, ack 5248096698, win 0, options [nop,nop,TS val 991525530 ecr 2000548351], length 0
00:00:00.000082 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [S], seq 2916099747, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 263287886 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.16177: Flags [S.], seq 423294695, ack 2916099748, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3174048641 ecr 263287886], length 0
00:00:00.000006 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], ack 423294696, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:00.000024 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], seq 2916099748, ack 423294696, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:00.000003 IP 127.0.0.1.5000 > 127.0.0.1.16177: Flags [F.], ack 2916099749, win 320, options [nop,nop,TS val 3174048641 ecr 263287886], length 0
00:00:00.000185 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], seq 423294696, ack 2916099749, win 320, options [nop,nop,TS val 3174048641 ecr 263287886], length 0
00:00:00.000083 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], ack 423294697, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:10.001400 IP 127.0.0.1.50346 > 127.0.0.1.5000: Flags [S], seq 863518883, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 212167698 ecr 0], length 0
00:00:00.000037 IP 127.0.0.1.5000 > 127.0.0.1.50346: Flags [S.], seq 1605084384, ack 863518884, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1433121945 ecr 212167698], length 0
00:00:00.000028 IP 127.0.0.1.50346 > 127.0.0.1.5000: Flags [F.], seq 1605084385, win 320, options [nop,nop,TS val 212167698 ecr 1433121945], length 0
00:00:00.000200 IP 127.0.0.1.50346 > 127.0.0.1.5000: Flags [R.], seq 1605084385, win 0, options [nop,nop,TS val 212167698 ecr 1433121945], length 0
[1] Done tcpdump -i lo0 -n -s0 -w exp6.pcap tcp port 5000
root@freebsd:~/col334_a2 #
```

Screenshot 6a — exp6.pcap readback: port 16177 performs a full four-way FIN close; port 50346 is terminated by a single RST.

```
root@freebsd:~/col334_a2 # tcpdump -i lo0 -n -s0 -w exp6.pcap 'tcp port 5000' &
root@freebsd:~/col334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
python3 experiment.py 6
=== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ===

Started Exchange Server (PID 11374).
  0.113ms listen fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  78.060ms accept sid=1 fd=5 peer='127.0.0.1:45901' nconn=1 sndbuf=49032 rcvbuf=81720

  78.102ms eof_rst sid=1 fd=5 errno=54 ev_fflags=54
Part A: orderly connection termination (FIN).
  78.142ms close sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
Orderly client: connected from 127.0.0.1:16177
The client will shut down its sending direction.
Investigate the TCP traffic and connection state.
  78.193ms destroy sid=1 fd=5 nconn=0
  78.295ms accept sid=2 fd=5 peer='127.0.0.1:16177' nconn=1 sndbuf=49032 rcvbuf=81720
  78.320ms eof_fin sid=2 fd=5 ev_eof=True
  78.333ms close sid=2 fd=5 why='FIN' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  78.355ms destroy sid=2 fd=5 nconn=0

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:50346
The client will now close abortively.
Investigate the TCP traffic and connection state.

The Exchange Server will remain running for 15 seconds.
10080.108ms accept sid=3 fd=5 peer='127.0.0.1:50346' nconn=1 sndbuf=49032 rcvbuf=81720
10080.312ms eof_rst sid=3 fd=5 errno=54 ev_fflags=54
10080.565ms close sid=3 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
10080.746ms destroy sid=3 fd=5 nconn=0

Experiment finished.
Stopping Exchange Server...
25084.603ms signal signo=15
```

Screenshot 6b — the server trace for the same run: eof_fin{ev_eof=True} for Part A against eof_rst{errno=54, ev_fflags=54} for Part B.

Part A (FIN), port 16177:

```
16177 > 5000: Flags [F.] <- client half-closes (shutdown SHUT_WR)
5000 > 16177: Flags [.] server ACKs
5000 > 16177: Flags [F.] <- server sends its OWN FIN back
16177 > 5000: Flags [.] client ACKs -- full four-way close
```

Server trace for the same connection:

```
78.295ms accept sid=2 fd=5 peer='127.0.0.1:16177' nconn=1
78.320ms eof_fin sid=2 fd=5 ev_eof=True
```

```
78.333ms close    sid=2 fd=5 why='FIN' recvs=0 bytes_in=0 queued=0 sent=0
```

Part B (RST), port 50346:

```
50346 > 5000: Flags [R.] seq ... win 0    <- single segment, no FIN exchange, no ACK
```

```
10080.108ms accept  sid=3 fd=5 peer='127.0.0.1:50346' nconn=1
```

```
10080.312ms eof_rst sid=3 fd=5 errno=54 ev_fflags=54
```

```
10080.565ms close   sid=3 fd=5 why='RST' recvs=0 bytes_in=0 queued=0 sent=0
```

Table T7

aspect	Part A — FIN	Part B — RST
wire	[F.], [F.], [F.], [F.] — four-way	single [R.]
recv() result	returns b''	raises ConnectionResetError
errno	n/a	54 (ECONNRESET)
ev.flags / ev.fflags	ev_eof=True, no error	ev_fflags=54
server trace	eof_fin → close(why='FIN')	eof_rst → close(why='RST')
TIME_WAIT observed?	not caught by polling	never — no wait state exists
other clients affected	none	none

The kqueue-specific observation. FreeBSD reports the socket error in `ev.fflags` on `KQ_FILTER_READ`, so **FIN and RST are distinguishable at the readiness-notification level, before a single byte is read**. In our traces `eof_rst` always carries `ev_fflags=54` and `eof_fin` never carries an error value — an asymmetry visible purely in the notification, which `select()` or `poll()` could not provide.

A documented design choice. In Part A the client closed only its *write* side and could legally still receive. Our server treats EOF as full teardown and sends its own FIN back immediately rather than draining `wbuf` on a half-open socket. That is why a complete four-way close appears rather than a lingering half-open connection. A half-close-aware server would differ here; we chose simplicity and documented it.

8. Experiment 7 — Backpressure and the Slow Receiver

This is the assignment's centrepiece, and it is also where our initial hypothesis was **wrong in an instructive way**. We report the full sequence because the correction is the result.

8.1 Answer

When a subscriber stops reading, its receive buffer fills, it advertises a zero window, the server's send buffer fills, and `send()` returns `EWouldBlock`. The server stops writing **to that socket only**, parks the backlog in that connection's userspace buffer, and re-arms `KQ_FILTER_WRITE` for it. Every other connection and the matching engine are untouched, because the engine never calls `send()`.

We confirmed the second half of that directly — **the engine is provably unaffected**. We could *not* confirm the first half by the expected route, and the reason turned out to be a property of the platform rather than of the server (§8.5).

8.2 Pre-flight arithmetic — stated before running

`b"TRADE INST 1 238\n"` is **17 bytes**; `TRADE_COUNT = 5000`, so each subscriber is offered **85,000 B** (measured: 85,003, the extra 3 being the `OK\n` reply to `SUBSCRIBE`).

Measured stock buffers: sendspace = 32768, recvspace = 65536 → combined capacity ≈ **98,304 B**.

98,304 > 85,003, so we predicted that at stock settings nothing would block. Reporting a null result you predicted is worth more than an unexplained one.

8.3 Run A — stock buffers

Prediction confirmed. send_would_block = **0** for the entire run; wbuf high-water mark **17 B** (one message) for both subscribers; queued == sent everywhere.

[illegible]

Screenshot 7a — six repeated sam-

ples of `netstat` and `ps -o wchan` across the run: the slow client's Recv-Q (port 36953) climbs steadily while the normal client (port 62129) stays near zero, and `wchan=kqread` on every sample.

The slow and normal subscribers are indistinguishable from the server's side — the divergence appears only in the clients' kernel receive queues:

sample	normal client Recv-Q (port 62129)	slow client Recv-Q (port 36953)
1	17	629
2	17	3,757
3	0	22,083
4	0	34,816
5	17	54,740
6	0	74,834
7	17	83,232

`wchan = kqread` on all seven samples (**W1**).

8.4 Runs B and D — shrinking the buffers, twice, with no effect

Run B reduced the buffers to force onset inside the window (`recvspace=8192 sendspace=4096`, both auto-tuning flags off; predicted onset at message ~723). **Every headline number came back identical to Run A.**

Run D capped the TCP-specific ceilings properly (`recvbuf_max=16384 sendbuf_max=16384`) and additionally set `SO_SNDBUF=1024` per socket via `setsockopt`, which also clears `SB_AUTOSIZE`. We verified the knobs took effect — the accept trace records `getsockopt` values per connection (`SO_SNDBUF=1024, SO_RCVBUF=8192`) and `netstat -x` confirms `S-HIWA=1024`. Predicted capacity 17,408 B, onset at message ~1024.

Still `send_would_block = 0`. The non-reading client's Recv-Q climbed monotonically **12,223 → 83,266** against its own R-HIWA of **8,192** — a tenfold overshoot on a freshly created connection.

A methodology casualty worth reporting. An intermediate attempt added `kern.ipc.maxsockbuf=65536`. That knob is the *global* ceiling for every socket on the machine, Unix-domain included; it starved `libcasper` and then `sshd`, which stopped completing handshakes. Recovery required the VM console. `net.inet.tcp.*` ceilings are safe to tune for an experiment; `kern.ipc.maxsockbuf` is not.

8.5 The correction: buffer limits are not enforced against a non-reading receiver

Because two independent attempts to shrink the target failed, we isolated the behaviour from the exchange entirely with a ~200-line blaster/victim pair (`src/tools/sb_probe.py`) containing no project code.

```

16777232      986896      1.43
17825792      1048576     1.52
18874369      1110257     1.62
19922946      1171938     1.71
20971523      1233619     1.80
22020100      1295300     1.88
23068677      1356981     1.97
24117254      1418662     2.06
25165831      1480343     2.15
26214408      1542024     2.23
27262985      1603705     2.32
28311562      1665386     2.41
29360139      1727067     2.50
30408716      1788748     2.59
31457293      1850429     2.68
32505870      1912110     2.78
33554447      1973791     2.87
34603024      2035472     2.96
35651584      2097152     3.05
36700161      2158833     3.13
37748738      2220514     3.21
38797315      2282195     3.30
39845892      2343876     3.39
40894469      2405557     3.48
41943046      2467238     3.57
42991623      2528919     3.65
44040200      2590600     3.75
45088777      2652281     3.84
46137354      2713962     3.93
47185931      2775643     4.02
48234508      2837324     4.11
49283085      2899005     4.21
50331662      2960686     4.30
51380239      3022367     4.38
52428816      3084048     4.46
53477376      3145728     4.54
54525953      3207409     4.63
55574530      3269090     4.72
56623107      3330771     4.81
57671684      3392452     4.89
58720261      3454133     4.97
59768838      3515814     5.05
60817415      3577495     5.13
61865992      3639176     5.21
62914569      3700857     5.30
63963146      3762538     5.39
65011723      3824219     5.47
66060300      3885900     5.56
67108877      3947581     5.65

=====
blaster RESULT
=====
PATH CAPACITY (bytes accepted before sustained stall) : 67108877
messages      : 3947581
first EWOULDBLOCK at : NEVER -- no backpressure at any point
EWOULDBLOCK count : 0
SO_SNDBUF in force : 49032

Capacity should be roughly own_sndbuf + peer_rcvbuf. If it is
orders of magnitude larger, the peer's sb_hiwat is not gating
acceptance -- which is the anomaly under investigation.

holding the connection open; Ctrl-C when the victim is done.
[

[root@freebsd:~/col334_a2 # python3 src/tools/sb_probe.py --client 127.0.0.1 6000
victim: connected. local_port=11085 SO_RCVBUF=81720 (requested None)
victim: NEVER calling recv(). Ctrl-C to drain and count.

elapsed_s      FIONREAD      SO_RCVBUF
0.0            0            81720
1.0            11728402     122580
2.0            23414032     122580
3.0            35118396     122580
4.0            47004930     122580
5.0            59126731     122580
6.0            67108877     122580
7.0            67108877     122580
8.0            67108877     122580
9.0            67108877     122580
10.0           67108877     122580
11.0           67108877     122580
12.0           67108877     122580
13.0           67108877     122580
14.0           67108877     122580
15.0           67108877     122580
16.0           67108877     122580
17.0           67108877     122580
18.0           67108877     122580
19.0           67108877     122580
20.0           67108877     122580
21.1           67108877     122580
22.1           67108877     122580
23.1           67108877     122580
24.1           67108877     122580
25.1           67108877     122580
26.1           67108877     122580
27.1           67108877     122580
28.1           67108877     122580
29.1           67108877     122580
30.1           67108877     122580
31.1           67108877     122580
32.1           67108877     122580
33.1           67108877     122580
34.1           67108877     122580
35.1           67108877     122580
36.1           67108877     122580
37.1           67108877     122580
38.1           67108877     122580
39.1           67108877     122580
40.1           67108877     122580
41.1           67108877     122580
42.1           67108877     122580
43.1           67108877     122580
44.1           67108877     122580
45.1           67108877     122580
46.1           67108877     122580
47.1           67108877     122580
48.1           67108877     122580
49.1           67108877     122580
50.2           67108877     122580
51.2           67108877     122580
52.2           67108877     122580
53.2           67108877     122580
54.2           67108877     122580
55.2           67108877     122580
56.2           67108877     122580
57.2           67108877     122580
58.2           67108877     122580
59.2           67108877     122580
60.2           67108877     122580

^C
victim: draining. FIONREAD said 67108877 readable.

=====
victim RESULT
=====
FIONREAD before drain : 67108877 B
ACTUALLY DRAINED      : 67108877 B
drain took             : 1.13 s
SO_RCVBUF start / end : 81720 / 122580
root@freebsd:~/col334_a2 #

```

Screenshot 7b — `sb_probe.py`, default-buffer run, both terminals: the victim's `FIONREAD` climbing to 67,108,877 with `SO_RCVBUF` autotuning 81,720 → 122,580, and the blaster's matching `PATH CAPACITY` result on the right — the live run reproduces the table below exactly.

	default buffer	SO_RCVBUF pinned to 8192
SO_RCVBUF start → end	81,720 → 122,580	8,192 → 8,192
bytes absorbed	67,108,877	67,108,877
overshoot vs. buffer	821×	8,192×
sustained stall	never	never
FIONREAD before drain	67,108,877	67,108,877
actually drained	67,108,877	67,108,877

Both runs stopped at the tool's own byte cap, **not at any kernel limit**. The receive buffer differed 15× between them and the absorbed byte count is identical to the byte. `FIONREAD` — read from inside the receiving process, never via `netstat` — agreed exactly with a drain-and-count, so the data genuinely was buffered.

The same tool on Linux stalls at **17,081 B** against `SO_SNDBUF 4,608 + peer SO_RCVBUF 16,384`, with `FIONREAD` flat at 12,985 — textbook flow control, and the behaviour FreeBSD is not showing on this path.

```
root@freebsd:~/col334_a2 # tcpdump -r exp7e.pcap -n 'src port 12398' | head -3
reading from file exp7e.pcap, link-type NULL (BSD loopback), snapshot length 96
18:24:54.285291 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [S], seq 3741972962, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2978424221 ecr 0], length 0
18:24:54.285430 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [.] , ack 226082803, win 320, options [nop,nop,TS val 2978424221 ecr 689218545], length 0
18:24:54.285476 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [P.] , seq 0:15, ack 1, win 320, options [nop,nop,TS val 2978424221 ecr 689218545], length 15
tcpdump: Unable to write output: Broken pipe
root@freebsd:~/col334_a2 # tcpdump -r exp7e.pcap -n 'src port 12398' | tail -3
reading from file exp7e.pcap, link-type NULL (BSD loopback), snapshot length 96
18:28:13.463490 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [.] , ack 11900004, win 320, options [nop,nop,TS val 2978623401 ecr 689417685], length 0
18:28:30.160716 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [F.] , seq 15, ack 11900004, win 320, options [nop,nop,TS val 2978640101 ecr 689417685], length 0
18:28:30.193722 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [.] , ack 11900005, win 320, options [nop,nop,TS val 2978640131 ecr 689434455], length 0
root@freebsd:~/col334_a2 #
```

Screenshot 7c — exp7e.pcap filtered to the subscriber's port (12398): the opening win 65535 at the SYN, settling to a constant win 320 at the first post-handshake ACK, and still win 320 at the very last packet — after 11,900,004 B absorbed unread.

What the wire shows (F3). exp7e.pcap contains **1,630,569** non-RST win 0 packets — and every one of them is server → trader (930,144 + 700,425, summing exactly). The win field is always the sender's own receive window, so these are the **server** throttling inbound order flow, not a subscriber throttling the server. From the non-reading subscriber: **zero**.

Better than a bare negative, that subscriber's window *never moved*:

```
SYN    12398 > 5000: win 65535, options [mss 16344, wscale 8, sackOK]
first  12398 > 5000: Flags [.] , win 320                <- 320 << 8 = 81,920 bytes
last   12398 > 5000: Flags [.] , win 320                <- after 11,900,003 B absorbed unread
```

A receiver whose buffer is filling should shrink progressively toward zero. This one advertised a constant, wide-open 81,920-byte window across 11.9 MB it never read. **The sender was never asked to slow down** — which is exactly why `send()` never returned `EWOULDLOCK`.

The control is inside the same capture: on the same kernel, the same loopback interface, in the same run, the server's TCP shrank *its* window to zero 1.6 million times. So this stack does perform receiver flow control. The failure is specific to the socket whose application never calls `recv()`.

What does bound it. `kern.ipc.nmbclusters 254,663` (~497 MiB), `kern.ipc.maxmbufmem ~1.94 GiB`. The 67 MiB run allocated ~208 MB of network memory — a 3.1× mbuf amplification that matches the independently measured `sb_mbcnt/sb_cc` ratio of 3.01 — i.e. ~10 % of budget, with requests for mbufs denied 0/0/0. We state this as a computed bound, not an observed one: confirming it would require driving the pool to exhaustion, and after the `maxsockbuf` incident we judged that unwise.

Four independent instruments agree: `netstat Recv-Q`, `getsockopt`, `FIONREAD` + `drain-and-count`, and the packet capture.

8.6 Run E — forcing backpressure by volume instead

Since the target could not be shrunk, we overwhelmed it. `exp7_load.py` pipelines 700,000 matched pairs with batched writes and no per-trade drain (the harness's own ~850 B/s pacing is what kept Runs A-D below any ceiling), draining the trader sockets continuously so they cannot become a second slow consumer. **Stock sysctls throughout — no tuning at all.**

```

root@freebsd:~/col334_a2 # ls -la ~/col334_a2/exp7e_trace.jsonl ~/col334_a2/exp7e.pcap
-rw-r--r--  1 root wheel 445086643 Sep  6 18:28 /root/col334_a2/exp7e.pcap
-rw-r--r--  1 root wheel 443652856 Sep  6 18:28 /root/col334_a2/exp7e_trace.jsonl
root@freebsd:~/col334_a2 # python3 src/tools/exp7_report.py exp7e_trace.jsonl
=====
Experiment 7 trace report: exp7e_trace.jsonl
=====

-- event counts -----
queue_out          3500003
engine             1400003
recv               360
send_would_block   66
accept             3
eof_fin            3
close              3
destroy            3
listen             1
write_enable       1
write_disable      1

-- connections (kernel buffers actually in force) -----
sid  fd  peer                SO_SNDBUF  SO_RCVBUF
1    6    127.0.0.1:12398        2048       81720
2    7    127.0.0.1:35156        2048       81720
3    8    127.0.0.1:54139        2048       81720

-- P1 backpressure onset -----
first send_would_block : t=93102.525 ms  sid=3
send_would_block/sid   : {3: 66}
partial_write/sid      : {}
engine calls before onset: 534524

-- P2 engine decoupling (engine_ns, microseconds) -----
window      n      p50      p90      p99      p99.9      max
pre-onset   534524  4.3      8.0      62.6     148.0     47054.2
post-onset  865479   3.9      6.6      55.0     121.0     64416.1

ratio post/pre      : p50 x0.91  p99 x0.88
ASSERTION: both ratios must be O(1). A ratio >> 1 would mean the
           matching engine slowed down once a consumer stopped
           reading, i.e. head-of-line blocking.

throughput (engine calls / s)
pre-onset      6550/s over 81.61 s
post-onset     7364/s over 117.52 s

-- P3 per-connection isolation -----
sid  queued_B  TRADEs(17B)  wbuf_hwm_B  max_pend
1    11900003   700000       17          17
2    28151893   10          23          23
3    26737003   0           1474        1474

ASSERTION: the non-reading subscriber shows wbuf_hwm >> 17 while
           every other sid stays at or near 17 (one message).
           That is per-connection isolation: the backlog is
           charged to the slow socket, not to the server.

-- close events -----
sid=1  why=FIN      queued=11900003  sent=11900003  eagain=0  hwm=17
sid=2  why=FIN      queued=28151893  sent=28151893  eagain=0  hwm=23
sid=3  why=FIN      queued=26737003  sent=26737003  eagain=66  hwm=1474

root@freebsd:~/col334_a2 #

```

Screenshot 7d — `exp7_report.py` re-run against the preserved Run E trace (`exp7e_trace.jsonl`, dated Sep 6 — the same run analyzed throughout this section, not a fresh one): onset, per-sid table, and the pre/post engine latency split, matching the figures and prose below exactly.

events: queue_out 3,500,003 · engine 1,400,003 · send_would_block 66
 write_enable 1 · write_disable 1
 onset: first send_would_block at t = 93,102.525 ms, sid=3

sid	role	queued (B)	wbuf_hwm	eagain
1	subscriber, never reads	11,900,003	17	0
2	buyer	28,151,893	23	0
3	seller	26,737,003	1,474	66

Where the backpressure landed, stated plainly. Not on the non-reading subscriber — on the *seller*. The load generator writes 500-line batches and drains the traders only between batches, so the server’s replies can briefly outrun its read loop. 66 events across 700,000 pairs (0.01 %) is a rare, self-correcting stall. It is genuine backpressure, correctly detected and handled, but it is a property of the generator’s write/drain interleaving rather than of a deliberately slow consumer, and we do not present it as the textbook case.

Artifacts P1 and P3. wbuf_hwm 1,474 B on sid 3 against 17 and 23 on the others is the per-connection isolation result: the backlog is charged to the connection that caused it and to no one else. But queued == sent at close for all three, so the backlog was *transient*, not standing — P3 therefore plots the difference series rather than two cumulative curves four orders of magnitude apart.

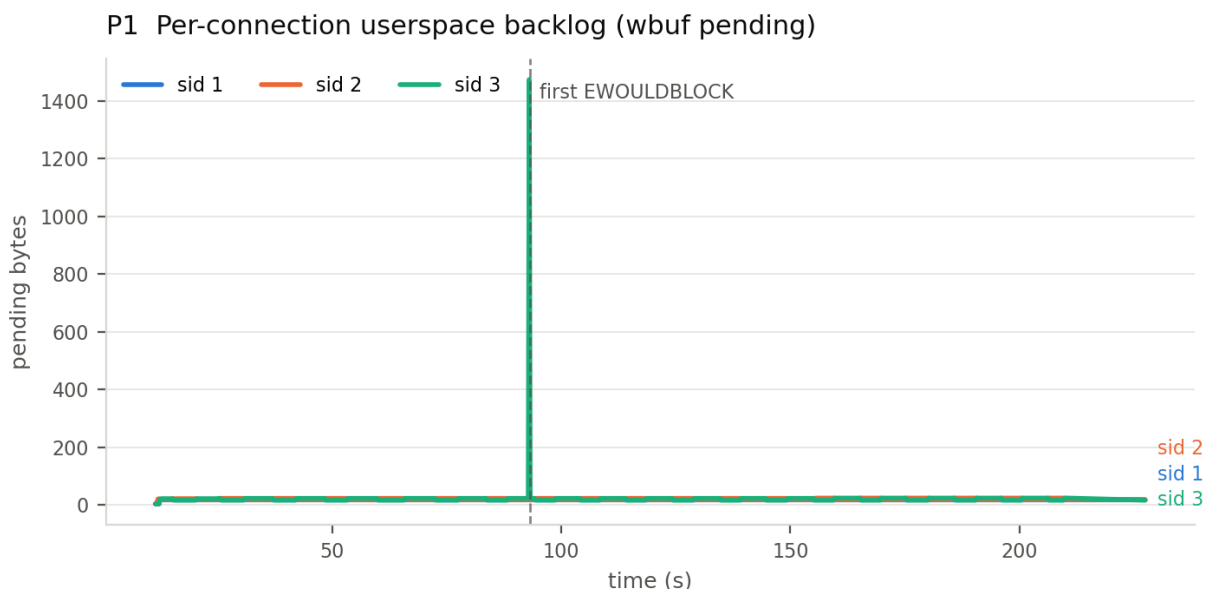


Figure P1 — per-connection userspace backlog vs time. The spike to 1,474 B on sid 3, aligned with the onset marker, is the only visible departure from a flat ~17-23 B baseline across all three connections.

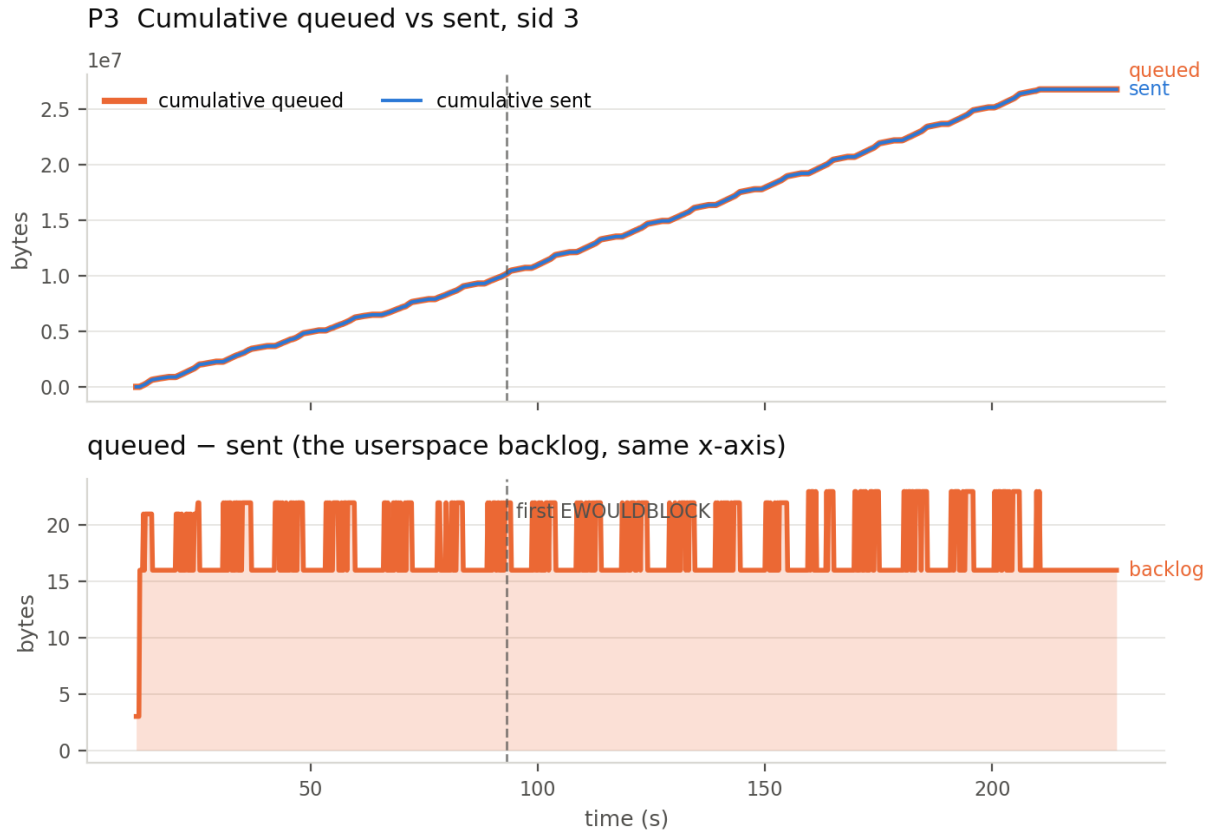


Figure P3 — cumulative queued vs sent for sid 3 (top panel, the two lines overlapping at this scale), and the queued–sent backlog difference on the same x-axis (bottom panel) — the same transient spike as P1, now shown against the 27 MB of total traffic it is a rounding error within.

8.7 The result that matters: the engine is decoupled

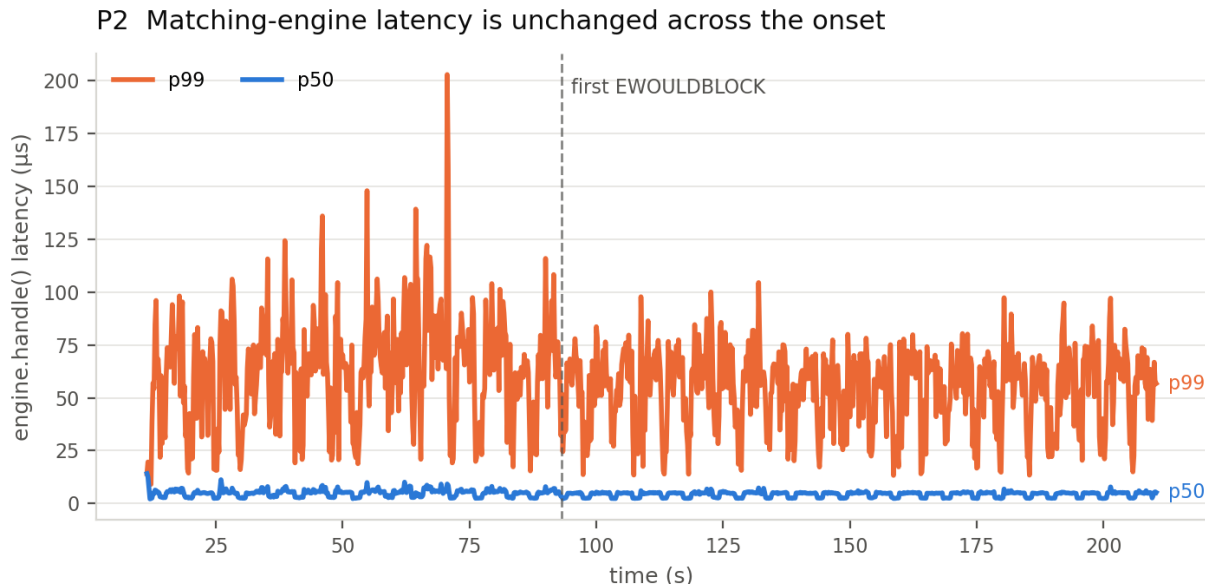


Figure P2 — matching-engine latency (p50, p99) across the full run. No step change at the onset line: the distribution before and after is the same shape, which is the positive evidence for decoupling, not merely the absence of a negative one.

window	n	p50	p90	p99	p99.9	max
pre-onset	534,524	4.3 µs	8.0	62.6	148.0	47,054
post-onset	865,479	3.9 µs	6.6	55.0	121.0	64,416

Ratios post/pre: **p50 ×0.91, p99 ×0.88** — the engine ran marginally *faster* after backpressure engaged. Throughput rose from **6,550/s to 7,364/s**. `write_enable` and `write_disable` each fired exactly once: the write filter was armed when the backlog appeared and correctly disarmed when it drained, so the level-triggered spin failure mode did not occur. `wchan` stayed `kqread` throughout.

A caveat we state rather than hide. `engine_ns` brackets only `engine.handle()`, so by construction it cannot include write-path cost. It proves the *matching engine* is unaffected; it does not by itself prove the event loop never stalled in the write path. Throughput (loop iterations per second) and `wchan` are what cover that. The claim needs all three.

8.8 What we would say if asked “did the experiment work?”

The pre-flight arithmetic predicted Run A’s null outcome correctly — but Runs B and D show it predicted the right answer for the *wrong reason*. Run A alone could not distinguish the two explanations, because both predict a null. Runs B and D could, and falsified the capacity model. We then redesigned around volume, obtained a genuine onset, and measured the decoupling the experiment exists to establish.

9. Experiment 8 — Unexpected Client Disconnection

Answer

SIGKILL is indistinguishable from `close()` on the wire. TCP has no concept of “the process died”: the kernel closes the dead process’s descriptors on exit, and for a TCP socket that means a perfectly normal FIN. Detection is therefore **reactive** — the server learns only on the next read or write of that socket. Our event loop, which watches every descriptor for readability, learned in microseconds.

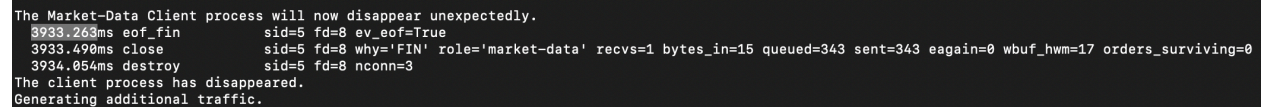
Approach

The harness SIGKILLS a real helper process, so the kernel — not the application — closes the socket. We correlate the server’s trace against the wire, then verify that the surviving clients were unaffected.

Commands and tools

```
tcpdump -i lo0 -n -s0 -w exp8.pcap 'tcp port 5000' &
python3 experiment.py 8
tcpdump -r exp8.pcap -n -ttt -S | grep 49686
```

Evidence



```
The Market-Data Client process will now disappear unexpectedly.
3933.263ms eof_fin      sid=5 fd=8 ev_eof=True
3933.490ms close       sid=5 fd=8 why='FIN' role='market-data' recvs=1 bytes_in=15 queued=343 sent=343 eagain=0 wbuf_hwm=17 orders_surviving=0
3934.054ms destroy     sid=5 fd=8 nconn=3
The client process has disappeared.
Generating additional traffic.
```

Screenshot 8a — the server’s own trace at the moment of the kill: `eof_fin` for `sid 5`, then `close{why='FIN', role='market-data', queued=343, sent=343, orders_surviving=0}` — nothing it was owed got truncated.

```

root@frebsd:~/col334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
kill %1
784 packets captured
784 packets received by filter
0 packets dropped by kernel
root@frebsd:~/col334_a2 # tcpdump -r exp8.pcap -n -ttt -S | grep 15960
reading from file exp8.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000784 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [S], seq 1521756197, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2869737745 ecr 0], length 0
00:00:00.000804 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [S.], seq 2819235974, ack 1521756198, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2140292229 ecr 2869737745], length 0
00:00:00.000813 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235975, win 320, options [nop,nop,TS val 2869737745 ecr 2140292229], length 0
00:00:00.000820 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 1521756198:1521756213, ack 2819235975, win 320, options [nop,nop,TS val 2869737745 ecr 2140292229], length 15
00:00:00.000830 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235975:2819235978, ack 1521756213, win 320, options [nop,nop,TS val 2140292229 ecr 2869737745], length 3
00:00:00.000785 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235978, win 320, options [nop,nop,TS val 2869737786 ecr 2140292229], length 0
00:00:00.000820 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235978:2819235995, ack 1521756213, win 320, options [nop,nop,TS val 2140293201 ecr 2869737786], length 17
00:00:00.0008571 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235995, win 320, options [nop,nop,TS val 2869738757 ecr 2140293201], length 0
00:00:00.000804 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235995:2819236012, ack 1521756213, win 320, options [nop,nop,TS val 2140293344 ecr 2869738757], length 17
00:00:00.0008992 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [.], ack 2819236012, win 320, options [nop,nop,TS val 2869738938 ecr 2140293344], length 0
00:00:00.000190 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236012:2819236029, ack 1521756213, win 320, options [nop,nop,TS val 2140293483 ecr 2869738938], length 17
00:00:00.0007780 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236029, win 320, options [nop,nop,TS val 2869739079 ecr 2140293483], length 0
00:00:00.0008044 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236029:2819236046, ack 1521756213, win 320, options [nop,nop,TS val 2140293624 ecr 2869739079], length 17
00:00:00.0043970 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236046, win 320, options [nop,nop,TS val 2869739186 ecr 2140293624], length 0
00:00:00.0008062 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236046:2819236063, ack 1521756213, win 320, options [nop,nop,TS val 2140293764 ecr 2869739186], length 17
00:00:00.0022395 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236063, win 320, options [nop,nop,TS val 2869739375 ecr 2140293764], length 0
00:00:00.0008065 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236063:2819236080, ack 1521756213, win 320, options [nop,nop,TS val 2140293907 ecr 2869739375], length 17
00:00:00.0008648 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236080, win 320, options [nop,nop,TS val 2869739461 ecr 2140293907], length 0
00:00:00.0008074 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236080:2819236097, ack 1521756213, win 320, options [nop,nop,TS val 2140294042 ecr 2869739461], length 17
00:00:00.0037495 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2819236097, win 320, options [nop,nop,TS val 2869739681 ecr 2140294042], length 0
00:00:00.0008057 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236097:2819236114, ack 1521756213, win 320, options [nop,nop,TS val 2140294183 ecr 2869739681], length 17
00:00:00.0008443 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236114, win 320, options [nop,nop,TS val 2869739738 ecr 2140294183], length 0
00:00:00.0008289 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236114:2819236131, ack 1521756213, win 320, options [nop,nop,TS val 2140294324 ecr 2869739738], length 17
00:00:00.0008222 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236131, win 320, options [nop,nop,TS val 2869739876 ecr 2140294324], length 0
00:00:00.0008039 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236131:2819236148, ack 1521756213, win 320, options [nop,nop,TS val 2140294472 ecr 2869739876], length 17
00:00:00.0008560 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236148, win 320, options [nop,nop,TS val 2869740026 ecr 2140294472], length 0
00:00:00.0008027 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236148:2819236165, ack 1521756213, win 320, options [nop,nop,TS val 2140294611 ecr 2869740026], length 17
00:00:00.0008585 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236165, win 320, options [nop,nop,TS val 2869740167 ecr 2140294611], length 0
00:00:00.0008041 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236165:2819236182, ack 1521756213, win 320, options [nop,nop,TS val 2140294755 ecr 2869740167], length 17
00:00:00.0008748 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], ack 2819236182, win 320, options [nop,nop,TS val 2869740311 ecr 2140294755], length 0
00:00:00.0008070 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2819236182:2819236199, ack 1521756213, win 320, options [nop,nop,TS val 2140294899 ecr 2869740311], length 17
00:00:00.0008743 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236199, win 320, options [nop,nop,TS val 2869740825 ecr 2140294899], length 0
00:00:00.0008064 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236199:2819236216, ack 1521756213, win 320, options [nop,nop,TS val 2140295041 ecr 2869740825], length 17
00:00:00.0049797 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236216, win 320, options [nop,nop,TS val 2869740609 ecr 2140295041], length 0
00:00:00.0008040 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236216:2819236233, ack 1521756213, win 320, options [nop,nop,TS val 2140295183 ecr 2869740609], length 17
00:00:00.0008281 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236233, win 320, options [nop,nop,TS val 2869740736 ecr 2140295183], length 0
00:00:00.0008387 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2819236233:2819236250, ack 1521756213, win 320, options [nop,nop,TS val 2140295319 ecr 2869740736], length 17
00:00:00.0045021 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236250, win 320, options [nop,nop,TS val 2869740883 ecr 2140295319], length 0
00:00:00.0008028 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236250:2819236267, ack 1521756213, win 320, options [nop,nop,TS val 2140295462 ecr 2869740883], length 17
00:00:00.0008761 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236267, win 320, options [nop,nop,TS val 2869741018 ecr 2140295462], length 0
00:00:00.0008060 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236267:2819236284, ack 1521756213, win 320, options [nop,nop,TS val 2140295608 ecr 2869741018], length 17
00:00:00.0008385 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236284, win 320, options [nop,nop,TS val 2869741160 ecr 2140295608], length 0
00:00:00.0008035 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236284:2819236301, ack 1521756213, win 320, options [nop,nop,TS val 2140295749 ecr 2869741160], length 17
00:00:00.0014690 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236301, win 320, options [nop,nop,TS val 2869741371 ecr 2140295749], length 0
00:00:00.0008042 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236301:2819236318, ack 1521756213, win 320, options [nop,nop,TS val 2140295893 ecr 2869741371], length 17
00:00:00.0008146 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236318, win 320, options [nop,nop,TS val 2869741448 ecr 2140295893], length 0
00:00:00.0008726 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 1521756215, ack 2819236318, win 320, options [nop,nop,TS val 2869741552 ecr 2140295893], length 0
00:00:00.0008021 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [.], ack 1521756214, win 320, options [nop,nop,TS val 2140296036 ecr 2869741552], length 0
00:00:00.0008884 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [F.], seq 2819236318, ack 1521756214, win 320, options [nop,nop,TS val 2140296036 ecr 2869741552], length 0
00:00:00.0008012 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236319, win 320, options [nop,nop,TS val 2869741552 ecr 2140296036], length 0
00:00:00.0008043 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 794015962:794015968, ack 2826931961, win 320, options [nop,nop,TS val 22855181 ecr 608618724], length 18
00:00:00.0008086 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 794015968, win 320, options [nop,nop,TS val 608618813 ecr 22855181], length 0
00:00:00.0008383 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2826931961:2826931976, ack 794015968, win 320, options [nop,nop,TS val 608618813 ecr 22855181], length 15
00:00:00.0008497 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 794015968:794015979, ack 2826931976, win 320, options [nop,nop,TS val 22855270 ecr 608618813], length 19
[1] Done tcpdump -i lo0 -n -s0 -w exp8.pcap tcp port 5000
root@frebsd:~/col334_a2 #

```

Screenshot 8b — exp8.pcap filtered to port 15960 (the killed client, per the accept sid=5 ... peer='127.0.0.1:15960' line): a normal four-way FIN close, indistinguishable from an orderly disconnect.

```

3933.263ms eof_fin sid=5 fd=8 ev_eof=True
3933.490ms close sid=5 fd=8 why='FIN' role='market-data' recvs=1 bytes_in=15
queued=343 sent=343 eagain=0 wbuf_hwm=17 orders_surviving=0
3934.054ms destroy sid=5 fd=8 nconn=3

15960 > 5000: Flags [.] ack ... <- last ACK of a TRADE push
15960 > 5000: Flags [F.] seq ... <- the SIGKILL'd process's FIN
5000 > 15960: Flags [.] ack ... server ACKs
5000 > 15960: Flags [F.] seq ... server's own FIN
15960 > 5000: Flags [.] ack ... client ACKs -- full four-way close

```

Table T10

time	event	wire	server observation	server
3933.263 ms	client SIGKILLED	[F.] from 15960	eof_fin{ev_eof=True}	mark d
+0.23 ms	—	[.] then [F.] from server	close{orders_surviving=0}	teardov
+0.56 ms	—	client [.]	destroy{nconn=3}	reap fd
through 8314 ms	trading continues	TRADE pushes to sid 2	—	unaffec

This is byte-for-byte the same shape as Experiment 6 Part A's orderly close — which is the point.

Why no EPIPE. The roadmap anticipated a possible FIN → data → RST → EPIPE sequence. It did not occur, and the trace explains why: detection via kevent() readiness beat any further

write attempt, so the server never tried to send() to a dead peer. Reaching EPIPE requires a write already in flight at the instant of death — a timing race this workload does not hit.

A gap in the harness we filled ourselves (T11). Experiment 8 kills a *Market-Data* client, which owns no orders — so it never exercises the order-survival requirement. We wrote the missing test: a trader logs in, rests BUY JNST 100 238, disconnects; a second trader then sells into it.

```
162192.955ms close sid=2 why='FIN' role='trader' orders_surviving=1
167024.123ms notify_dropped target_sid=2 msg=b'BOUGHT JNST 60 238\n'
```

Client-side: the surviving seller received SOLD JNST 60 238, the subscribed Market-Data client received TRADE JNST 60 238, and only the departed buyer's private BOUGHT was dropped. **The trade executed; the order outlived its connection.**

An asymmetry worth naming. A departing *subscriber* is removed from subs[instrument] at close time, so later broadcasts simply never iterate over it — no dropped notification is logged. A departing *trader's resting order* is deliberately left in the book, so a later match does still attempt the notification and does produce a logged notify_dropped. Subscription cleanup is immediate and silent; order survival is deliberate and observable.

10. Additional observations

10.1 Coalescing — the reverse of Experiment 3

Experiment 3 shows one message arriving as four segments. The complement also holds: in a supplementary run, a trader's LOGIN plus three SELL lines arrived in a **single** segment and were separated only by the framer.

```
recv sid=1 fd=6 n=67 rbuf_before=0 rbuf_after=0 lines_out=4
recv sid=2 fd=6 n=31 rbuf_before=0 rbuf_after=0 lines_out=2
```

Together, these bracket the byte-stream property from both sides: application message boundaries and TCP segment boundaries are independent in *both* directions.

A prediction of ours that was wrong. We expected the server's *outbound* replies (ORDER_ACCEPTED + 3×BOUGHT) to coalesce into one segment. They do not — tcpdump -A shows three separate 19-byte segments. The trace explains it: pending reads 17, 19, 19, 19 and never accumulates, because queue_out() flushes after every message and TCP_NODELAY prevents Nagle from re-merging them. Outbound coalescing *does* occur, but only under backpressure, which is precisely what Run E's wbuf_hwm = 1,474 is. The behaviour is the right one: per-message flush minimises latency while the peer keeps up, and batches automatically when it does not.

10.2 errno reference (T12)

errno	name	status	evidence, or why not
35	EAGAIN/EWOULDBLOCK	observed	Run E: 66 send_would_block events, eagain=66 at close. Raised as BlockingIOError, caught in flush(), answered by arming KQ_FILTER_WRITE.
54	ECONNRESET	observed, abundantly	Every experiment — the harness's SO_LINGER{1,0} readiness probe yields eof_rst errno=54 ev_fflags=54; Experiment 6 Part B is the deliberate case.
32	EPIPE	not observed	Requires a write in flight at the instant of peer death; kevent()-driven detection pre-empts it (§9). Python ignores SIGPIPE, so it would surface as BrokenPipeError, which flush() already handles.
60	ETIMEDOUT	not observed	Requires a retransmission timeout — a peer that stops responding without RST. Impossible on lo0, which has no loss and no path to lose a peer silently.

10.3 Market-Data client commands, completed

§2.8 lists SUBSCRIBE, UNSUBSCRIBE and QUIT as valid Market-Data Client messages, but the original client only ever read its socket — it could send none of them after the initial connect-time SUBSCRIBE. market_data.py now multiplexes stdin alongside the socket with the same select.kqueue() pattern trader.py already used, so UNSUBSCRIBE <instr> and QUIT can be typed live. Server-side support for both was already correct and unused; this closed the client-side gap. Verified offline: SUBSCRIBE+UNSUBSCRIBE round-trips two OKs and removes the session id from engine.subs; SUBSCRIBE+QUIT acknowledges first and then closes, per §2.8.

10.4 Measurement methodology

Instruments split into two classes, and only one class is timing-sensitive:

- **Live** — netstat, sockstat, ps -o wchan, getsockopt, FIONREAD. These must be sampled *during* a run, and all of ours were.
- **Frozen** — .pcap files, the JSONL trace, console logs. tcpdump -r reads a file, so re-reading later is a pure function of its contents; this also makes the analysis reproducible by anyone holding the same capture.

Run E's capture reported 5252898 captured / 5252898 received by filter / 0 dropped by kernel, so no packets were missed.

One reduction bug is worth recording because a figure exposed it: our first plot of P1 sampled the *last* pending value per time bucket, but pending is traced after the append and before the flush, so in the unblocked case it is merely the message size — which averaged the backpressure spike away entirely. Carrying a per-bucket maximum fixed it. The instrumentation was correct; the reduction was not.

11. Reproducing this work

src/server.py	the Exchange Server (the only server file importing socket)
src/framing.py	newline framing — no socket import
src/protocol.py	parsing and validation — no socket import
src/engine.py	order book and matching — no socket import
src/trader.py	Trader Client (kqueue-multiplexed stdin + socket)
src/market_data.py	Market-Data Client
tests/test_all.py	29 offline tests, including the fragmentation proofs

Instrumentation is enabled by environment variable, since the harness fixes argv:

EXCH_TRACE=<path>	structured JSONL trace, used for the figures
EXCH_SNDBUF=<bytes>	SO_SNDBUF on accepted sockets (Experiment 7 control)
EXCH_WBUF_MAX=<bytes>	slow-consumer backlog cap (default 4 MiB)
EXCH_NAIVE=1	launch the blocking control server (Experiment 4 only)

Non-submission analysis tools live in src/tools/: exp7_load.py (volume generator), exp7_report.py (trace reducer), exp7_probe.py and sb_probe.py (the isolation diagnostics), exp7_csv.py + plot.py (figures), verify_quit.py (§2.2.5 offline proof), gapd_concurrency.sh (§4.3's ≥ 10 -client / ≥ 2 -trader / ≥ 4 -MD requirement).